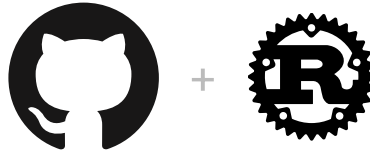


A Transpiler from C to Dafny

VectorPikachu

January 28, 2025



vectorpikachu/**Transpiler-from-C-to-Dafny**

A simple transpiler from C to Dafny.

Contents

1. Basic Usage	2
2. Translation Details	2
2.1. Arrays	2
3. Soundness Proof	3

1. Basic Usage

本项目使用 Rust 编写，使用 cargo 进行构建. 首先，确保你已经安装了 Rust 和 cargo.

```
1 cargo run -- -i <input_file> -o <output_file> bash
```

2. Translation Details

我提供了一个 `traverse_tree()`, 它可以遍历语法树并打印出每个节点的类型. 我们可以通过这个函数来查看语法树的结构.

- `preproc_include` 类型是头文件
- `preproc_function_def` 类型是宏中的函数定义. e.g. `preproc_function_def`:
#define assume(e) if(!(e)) exit(-1);
- `preproc_def` 类型是宏定义. e.g. `preproc_def`: #define a (2)
- `function_definition` 类型是函数定义. e.g. `function_definition`: int main()
{ return 0; }

我为每个表达式手动标记了类型, 以试图捕捉 C 语言中的强制类型转换, 并将其转换为 Dafny 中的 `as` 操作符. 我们可以在打印的时候加上这些操作符.

2.1. Arrays

关于数组: `seq<T>` 是 Dafny 中一个更加基本的类型, 而且虽然没有 `old` 操作符的支持, 但是我们可以通过下面的形式来模拟:

```
1 ghost var prevElements := a[..]; Dafny
2 while // ...
3   invariant a[lo..hi] == prevElements[lo..hi]
4 {
5   // ...
6 }
```

下面是 Deepseek-R1 给出的对比:

特性	seq（不可变序列）	array（可变数组）
可变性	不可变	可变
验证复杂度	低（自动推理友好）	高（需手动维护不变式）
内置操作支持	丰富（切片、拼接等）	基础（依赖索引访问）
适用场景	数据无需修改或频繁生成新序列	需要高效地修改数据
验证性能	快（状态变化少）	慢（需跟踪每一步修改）

下面是 GPT-4o 给出的对比:

特性	seq	array
可变性	不可变，适合函数式编程和数学证明	可变，适合实际程序设计和性能优化

抽象层次	高层次，贴近数学概念	低层次，贴近底层实现
性能	操作产生新序列，开销较高	原地更新，性能更高
证明难度	不需要管理状态，证明更简单	需要处理状态变化，证明复杂
适用场景	形式化证明、不可变数据的操作	高性能计算、频繁修改数据的程序

综上所述, 我决定使用 `seq` 类型来模拟数组.

在 [Dafny Reference Manual](#) 中, 我们可以看到 `seq` 类型的一些使用:

operator or expression	result type	description
<code><</code>	<code>bool</code>	proper prefix
<code><=</code>	<code>bool</code>	prefix
<code>+</code>	<code>seq<T></code>	concatenation
<code> s </code>	<code>nat</code>	sequence length
<code>s[i]</code>	<code>T</code>	sequence selection
<code>s[i := e]</code>	<code>seq<T></code>	sequence update
<code>e in s</code>	<code>bool</code>	sequence membership
<code>e !in s</code>	<code>bool</code>	sequence non-membership
<code>s[lo..hi]</code>	<code>seq<T></code>	subsequence
<code>s[lo..]</code>	<code>seq<T></code>	drop
<code>s[..hi]</code>	<code>seq<T></code>	take
<code>s[slices]</code>	<code>seq<seq<T>></code>	slice
<code>multiset(s)</code>	<code>multiset<T></code>	sequence conversion to a multiset<T>

我需要重新改造我的 `ast` 设计.

3. Soundness Proof

假设我们有这样几个转换:

- T_p : 从 C 的程序片段到 Dafny 的程序片段
- T_c : 从 C 的规约到 Dafny 的规约

对于任意 C 的霍尔三元组 $\langle c_1, p, c_2 \rangle$, 其对应的 Dafny 三元组为 $\langle T_c(c_1), T_p(p), T_c(c_2) \rangle$.

Soundness 定义为如果后者成立, 那么前者一定成立.

1. 现在假设 p 中出现了 `int` 类型, 并且出现了 `int` 的溢出操作, 那么 p 后面进行什么操作都是可以的, 因为这是一个 UB, 所以 c_2 无论如何都将成立, 所以我们可以直接把 C 语言中的 `int` 建模为 Dafny 中的 `int` 类型.
2. 如果 p 中出现了 `unsigned char` 和 `unsigned int`, 他们的溢出操作是有定义的回绕, 所以必须是 `bv` 类型.